Map-Reduce Data Processing

Map-Reduce Data Processing

- Generating a single value from a dataset through *map* and *reduce* operations.
  - *Map* operation (intermediate operation)
    - Transforms an input dataset to another dataset.
    - e.g., `map()`, `flatMap()`
  - *Reduce* operation (terminal operation)
    - Processes the transformed dataset to generate a *single* value.
    - e.g. `count()`, `max()`, `min()`

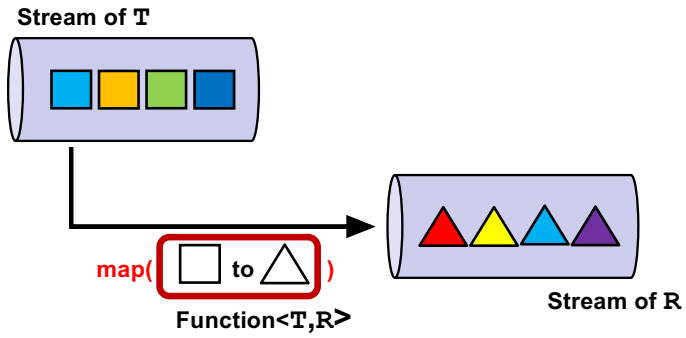
Map Operation

- `map(Function<T,R>)` : *intermediate* operation
  - Performs a stream-to-stream transformation
    - Takes a `Function` that converts a value of `T` to another of `R`.
      - `T` and `R` can be different types.
    - Applies the function on stream elements one by one.
    - Returns another stream of new values.
      - The # of elements does NOT change in b/w the input and output streams.

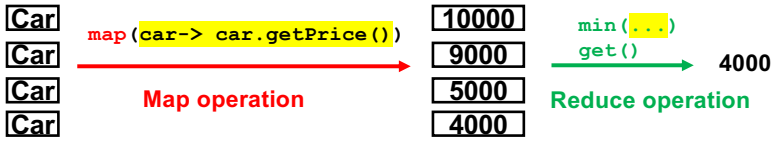
```
List<String> collected =
                Stream.of("u", "m", "b")
                      .map((String str)-> str.toUpperCase())
                      .collect( Collectors.toList() );
  // a list of "U", "M" and "B" is returned.
```

	Params	Returns	Example use case
Function<T,R>	T	R	Get the price (R) from a Car object (T)

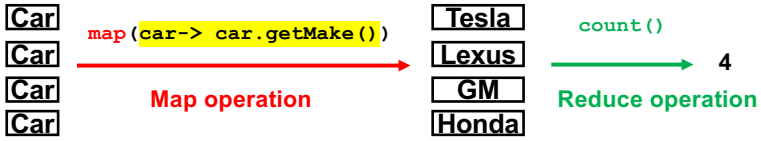
	Params	Returns	Example use case
Function<T,R>	T	R	Get the price (R) from a Car object (T) Generate a function (R) from another (T)



```
Integer price = cars.stream()
                .map( (Car car)-> car.getPrice() )
                .min( Comparator.comparing((Integer price)-> price ) )
                .get();
```



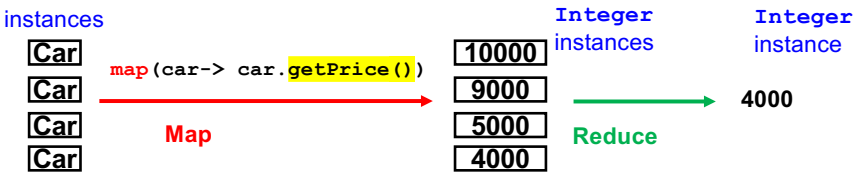

```
long carMakerNum = cars.stream()
                   .map( (Car car)-> car.getMake() )
                   .count();
```



Auto-boxing and Auto-unboxing

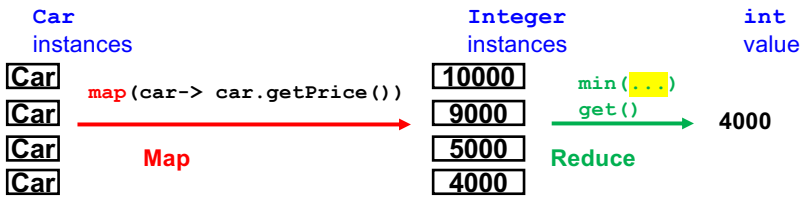
- Stream API performs auto-boxing and auto-unboxing whenever necessary and appropriate.

```
Integer price = cars.stream()
                .map( (Car car)-> car.getPrice() )
                ...
```



  - Even though `getPrice()` returns an `int` value, `map()` auto-boxes it to an `Integer` instance and creates a stream of `Integer` instances.
    - Reference types are used for stream elements, by default.


```
int price = cars.stream()
            .map( (Car car)-> car.getPrice() )
            .min( Comparator.comparing((Integer price)-> price ) )
            .get();
```



  - Even though `get()` return an `Interger` instance, it is auto-unboxed to an `int` value.

-
```
Stream<Double> randomNums = Stream.generate( ()-> Math.random() );
// infinite sequence of random numbers are generated.
```

  - Even though `random()` returns a `double` value, `generate()` auto-boxes it to a `Double` instance and creates a stream of `Double` instances.
    - Reference types are assumed for stream elements, by default.

-
```
Stream<Integer> integers =
                    Stream.iterate(0, (Integer i)-> i + 1);
// Infinite sequence of 0, 1, 2, 3...
```

Stream of Native-type Values

- You can create and use a stream of native-type values, if you want,
  - rather than a stream of class instances.
  - `IntStream` for a stream of `int` values
  - `LongStream` for a stream of `long` values
  - `DoubleStream` for a stream of `double` values

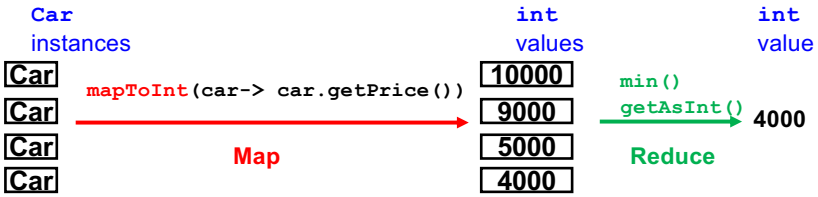
- These "special" streams implement useful methods, which are not available in `Stream<T>`.
  - `sum()`, `average()`
  - `min()`, `max()`, `sorted()`: Do not take a `Comparator`.
  - `summaryStatistics()`: Returns a stats summary.

- Call `of()` to create a stream of native-type values from scratch.

```
double avg = IntStream.of(1, 2, 3)
                      .average()
                      .orElse(0);
```

- Call `mapToInt()`, `mapToLong()` or `mapToDouble()` on an existing stream to get a stream of native-type values.

```
int minPrice = cars.stream()
                   .mapToInt( (Car car)-> car.getPrice() )
                   .min()
                   .getAsInt();
```



- `import static java.util.Arrays.*;`

```
List<Integer> vals = asList(1, 2, 3, 4, 5);
IntSummaryStatistics stats = vals.stream()
                                 .mapToInt(Integer::intValue)
                                 .summaryStatistics();

stats.getMax();
stats.getAverage();
...
```

- Call `boxed()` on an existing stream of native-type values to get a stream of wrapper instances.

```
List<Integer> list = IntStream.of(1, 2, 3)
                              .boxed()
                              .collect(Collectors.toList());
```


Recap: Method References

- *instance::method*
  - `System.out::println`
    - `System.out` contains an instance of `PrintStream`.
  - `(int x) -> System.out.println(x)`

- *Class::staticMethod*
  - `Math::max`
  - `(double x, double y) -> Math.max(x, y)`

- *Class::method*
  - `Car::getPrice`
  - `(Car car)-> car.getPrice()`

  - `Car::setPrice`
  - `(Car car, int price)-> car.setPrice(price)`

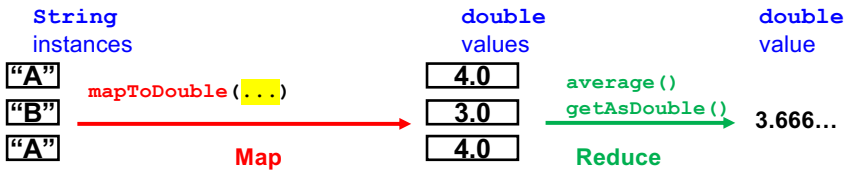
Exercise: GPA Calculation

```
HashMap<String, String> transcript = new HashMap<>();
transcript.put("CS680", "A");
transcript.put("CS681", "B");
transcript.put("CS682", "A");
```

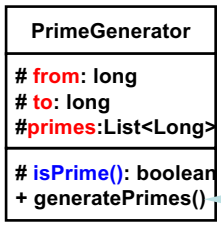


```
HashMap<String, String> transcript = new HashMap<>();
transcript.put("CS680", "A");
transcript.put("CS681", "B");
transcript.put("CS682", "A");

double gpa = transcript.values()
              .stream()
              .mapToDouble((String grade)->{
                  if(grade.equals("A")){return 4.0;}
                  else if(...){...}
                  else if ... })
              .average()
              .getAsDouble();
```

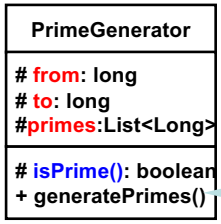


Exercise: Prime Number Generation



Without Stream API (c.f. CS680)

```
for(long n = from; n <= to; n++) {
        if( isPrime(n) ){ primes.add(n); } }
```



With Stream API

```
primes = LongStream.rangeClosed(from, to)
                   .filter((long n)->isPrime(n))
                   ...}
```


- **range**(long startInclusive, long endExclusive)
  - Create a stream from `startInclusive` (inclusive) to `endExclusive` (exclusive) by an incremental step of 1.

- **rangeClosed**(long startInclusive, long endInclusive)
  - Create a stream from `startInclusive` (inclusive) to `endInclusive` (inclusive) by an incremental step of 1.

Exercise: Order Queries

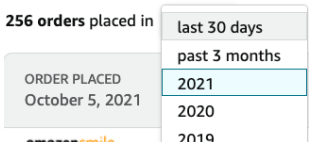
- Suppose an `Account` manages a list of `Order`s.
- How to write `Account`'s methods with Stream API?
  - `getOrdersInYr(int year)`

```
return orders.stream()
             .filter( ... )
             .collect(collectors.toCollection(LinkedList::new));
```

  - `getTotalExpenditureInYr(int year)`

```
return this.getOrdersInYr(year).stream()
                               .mapToDouble( ... )
                               .sum();
```

- FYI:
  - `LocalDate today = LocalDate.now();`
  - `int year = today.getYear();`



Order
- date: LocalDate - total: double
+ getDate(): LocalDate + getTotal(): double

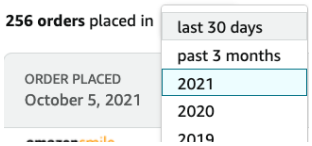
Account
- orders: LinkedList<Order>
+ getOrdersInYr(year: int): LinkedList<Order> + getTotalExpenditureInYr(year:int): double

```
getOrdersInYr(int year):
  return orders.stream()
               .filter( (order)-> order.getDate().getYear()==year )
               .collect(Collectors.toCollection(LinkedList::new));

getTotalExpenditureInYr(int year):
  return this.getOrdersInYr(year).stream()
                                 .mapToDouble((order)-> order.getTotal())
                                 .sum();
```


```
getTotalExpenditureInYr(int year):
  return orders.stream()
               .filter( (order)-> order.getDate().getYear()==year )
               .mapToDouble( (order)-> order.getTotal() )
               .sum();
```

- FYI:
  - `LocalDate today = LocalDate.now();`
  - `int year = today.getYear();`



Order
- date: LocalDate - total: double
+ getDate(): LocalDate + getTotal(): double

Account
- orders: LinkedList<Order>
+ getOrdersInYr(year: int): LinkedList<Order> + getTotalExpenditureInYr(year:int): double

Reduce Operations

- `Stream` provides…
  - "ready-made" methods for simple data processing tasks.
    - e.g. `count()`, `max()`, `min()`, `sum()`, `average()`
  - `collect()` to perform more complex data processing tasks.
  - `reduce()` to implement your own (custom) reduce operation..

Reduce Operations with `collect()`

- `collect(Collector)`: *terminal* operation
  - Takes a `Collector`,
  - Uses the `Collector` to transform/reduce stream elements to a value of a particular type (e.g. collection class) , and
  - Returns that value.

```
List<String> collected = Stream.of("u", "m", "b")
                               .map((String str)-> str.toUpperCase())
                               .collect( Collectors.toList() );
// a list of "U", "M" and "B" is returned.
```

  - `Collectors` implements various static factory methods that create and return `Collector` objects.
    - `Collectors.toList()`
      - Returns a `Collector` object that collects stream elements and transforms them to a `List`.

- **`Collectors`** implements various static factory methods that create and return `Collector` objects.
  - `Collectors.toList()`
    `Collectors.toSet()`
    - Returns a `Collector` object that collects stream elements as a `List` or a `Set`.
  - `Collectors.toCollection()`
    - Returns a `Collector` object that collects stream elements as a specific collection.
      - `Collectors.toCollection(ArrayList::new)`
      - `Collectors.toCollection(LinkedList::new)`
      - `Collectors.toCollection(HashSet::new)`

      ```
      ArrayList<Car> carArrayList =
          cars.stream()
              .collect( toCollection(ArrayList::new) );
      ```


  - `Collectors.toMap( Function<T,R>, Function<T,U> )`
    - A returned `Collector` collects stream elements of `T` and transforms them to a `Map<R,U>`.

    ```
    Map<Integer, String> idToAutoMaker =
        cars.stream()
            .collect( Collectors.toMap( (Car car)-> car.getId(),
                                        (Car car)-> car.getMake()));
    ```

      – Transforms a stream of `Car`s to a `Map<Integer,String>`

    ```
    Map<Integer, Car> idToCar =
        cars.stream()
            .collect( Collectors.toMap( (Car car)-> car.getId(),
                                        (Car car)-> car ));
    ```

      – Transforms a stream of `Car`s to a `Map<Integer,Car>`

  - `toMap()` throws an exception if keys collide.
    - Can take a function to handle potential key collisions.

    ```
    Map<Integer, String> idToAutoMaker =
        cars.stream().collect(Collectors.toMap(
            (Car car)-> car.getId(),
            (Car car)-> car.getMake(),
            (existingVal, newVal)-> existingVal + newVal ));
    ```

  - `toMap()` can return a `Collector` object that collects stream elements as a specific collection.

    ```
    HashMap<Integer, Car> idToCar =
        cars.stream().collect(Collectors.toMap(
            (Car car)-> car.getId(),
            (Car car)-> car,
            (existingVal, newVal)-> newVal,
            HashMap::new ));
    ```


Tip: Static-import of `Collectors`

- It is bothersome to write "`Collectors`" here and there many times.
- Static-import it.
  - `import static java.util.stream.Colletors.*;`
  - With the static import, you can do:

    ```
    collect(toList())
    ```

    rather than:

    ```
    collect(Collectors.toList())
    ```

  - Recall you did this for `Assertions` in CS680.

Extra Methods of `Collectors`

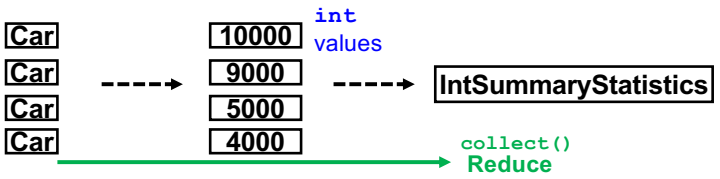
- **`Collectors`** is not only for stream-to-collection conversions.
  - Provides many more useful methods.

- **`joining()`**
  - Performs a stream-to-string conversion.
  - Returns a `Collector` object that concatenates stream elements into a string, separated by a given delimiter.

```
String concat = cars.stream()
                    .map( (Car car)-> car.getId() )
                    .collect(joining(", "));
```


- **`summarizingInt(ToIntFunction<T> mapper)`**
  - Returns a `Collector` object that applies a given LE (int-producing map operation) to each stream element and returns summary statistics for those values.

```
IntSummaryStatistics stats =
    cars.stream()
        .filter((Car car)->!car.hadAccidents())
        .collect(summarizingInt((Car car)->car.getPrice()));
stats.getAverage();
```



  - **`IntSummaryStatistics`** has:
    - **`getAverage()`, `getCount()`, `getMax()`, `getMin()`, `getSum()`**
    - c.f. **`DoubleSummaryStatistics`, `LongSummaryStatistics`**

- **`collectingAndThen(Collector, Function)`**
  - Useful to do some extra task on a collected result.

```
int averagePrice =
        cars.stream()
            .filter((Car car)->!car.hadAccidents())
            .collect(collectingAndThen(
                  summarizingInt((Car car)->car.getPrice()),
                  (stats)->stats.getAverage()));
```


`Comparator.thenComparing()`

- **`Comparator.comparing()`**
  - Higher-order function; c.f. CS680.
    - Takes a key extraction function, which extracts a sort key.
    - Returns a comparison function.

```
List<Car> sortedCars =
  cars.stream()
      .sorted( Comparator.comparing((Car car)-> car.getPrice()) )
      .collect( Collectors.toList() );
```

- **`Comparator.thenComparing()`**

```
List<Car> sortedCars =
  cars.stream()
      .sorted( Comparator.comparing((Car car)-> car.getPrice())
                         .thenComparing(car-> car.getMileage()) )
      .collect( Collectors.toList() );
```